

Modul 295 - Aufgabenblatt 05

von Lukas Meier

Unterricht vom 08.09.2026

1 Projekt vorbereiten

Erstellen Sie im `htdocs`-Ordner Ihrer XAMPP-Installation einen neuen Ordner `php-funktionen` und speichern Sie die Datei `index.php`, welche Sie auf dem Netzwerkordner finden, darin. Öffnen Sie danach `http://localhost/php-funktionen/` im Browser.

2 Erste Funktion

Schreiben Sie eine Funktion `greet()` ohne Parameter, die mit `echo` den Text "Hallo zusammen!" ausgibt. Rufen Sie die Funktion zweimal auf.

3 Funktion mit Parameter

Schreiben Sie eine Funktion `greetByName($name)` mit einem Parameter `$name`, die "Hallo <Name>!" ausgibt. Rufen Sie die Funktion mit zwei verschiedenen Namen auf.

4 Funktion mit Rückgabewert

Schreiben Sie eine Funktion `add($a, $b)`, die zwei Zahlen mit `return` addiert zurückgibt (statt sie direkt auszugeben). Speichern Sie das Ergebnis eines Aufrufs in einer Variable und geben Sie diese mit `echo` aus. Verwenden Sie einen zweiten Aufruf direkt in einer Berechnung, z.B. `echo add(10, 5) * 2;`

5 Funktion mit mehreren Parametern

Schreiben Sie eine Funktion `calculateRectangleArea($width, $height)`, die die Fläche eines Rechtecks berechnet und zurückgibt. Testen Sie die Funktion mit zwei unterschiedlichen Wertepaaren und geben Sie die Ergebnisse mit `echo` aus.

6 Default-Werte

Erweitern Sie eine Begrüßungsfunktion `greetWithGreeting($name, $greeting = "Hallo")` um einen zweiten Parameter `$greeting` mit dem Default-Wert "Hallo". Rufen Sie die Funktion einmal nur mit dem Namen und einmal zusätzlich mit einer eigenen Begrüßung (z.B. `SServus`) auf.

7 Type Hints

Schreiben Sie eine Funktion `multiply(int $a, int $b): int`, welche zwei Ganzzahlen multipliziert. Geben Sie das Ergebnis einmal mit `echo` und einmal mit `var_dump()` aus und prüfen Sie, dass der Rückgabetyt tatsächlich `int` ist.

8 Benannte Argumente

Rufen Sie die Funktion `greetWithGreeting()` aus Aufgabe 6 zweimal mit **benannten Argumenten** (`name: ..., greeting: ...`) auf - einmal in der ursprünglichen Reihenfolge und einmal mit vertauschter Reihenfolge der Argumente. Prüfen Sie, dass beide Aufrufe dasselbe Ergebnis liefern.

9 Variable Anzahl Argumente

Schreiben Sie eine Funktion `sum(...$numbers)`, die mit dem Variadic-Operator `...` eine beliebige Anzahl Zahlen entgegennimmt und deren Summe mit einer `foreach`-Schleife berechnet und zurückgibt. Testen Sie die Funktion mit unterschiedlich vielen Argumenten.

10 Array zurückgeben

Schreiben Sie eine Funktion `minMax($numbers)`, die zu einem übergebenen Zahlen-Array den kleinsten und den grössten Wert (mit `min()` und `max()`) als Array `[$min, $max]` zurückgibt. Rufen Sie die Funktion einmal auf und greifen Sie über den Index auf beide Werte zu. Rufen Sie sie ein zweites Mal auf und weisen Sie das Ergebnis direkt zwei Variablen `$smallest` und `$largest` zu (`[$smallest, $largest] = ...`).

11 Gültigkeitsbereich (Scope)

Legen Sie eine globale Variable `$counter = 0`; an. Schreiben Sie eine Funktion `incrementLocal()`, die *innerhalb* der Funktion eine eigene, lokale Variable `$counter` auf 0 setzt und um 1 erhöht. Rufen Sie sie zweimal auf und geben Sie danach die globale `$counter` aus - sie bleibt bei 0. Schreiben Sie danach eine zweite Funktion `incrementGlobal()`, die mit dem Schlüsselwort `global` auf die globale `$counter` zugreift und sie erhöht. Rufen Sie auch diese zweimal auf und geben Sie `$counter` erneut aus - dieses Mal hat sich der Wert verändert.

12 Statische Variable

Schreiben Sie eine Funktion `countCalls()` mit einer `static`-Variable `$count`, die bei jedem Aufruf um 1 erhöht wird und den Text "Aufruf Nr. X" ausgibt. Rufen Sie die Funktion dreimal auf und beobachten Sie, dass sich der Zähler den Wert zwischen den Aufrufen merkt.

13 Pass by Reference

Schreiben Sie zwei Funktionen: `addOne($n)`, die den Parameter normal (*by value*) entgegennimmt und um 1 erhöht, sowie `addOneByRef(&$n)`, die den Parameter *by reference* entgegennimmt. Legen Sie eine Variable `$x = 5`; an, rufen Sie zuerst `addOne($x)` auf und geben Sie `$x` aus (unverändert), rufen Sie danach `addOneByRef($x)` auf und geben Sie `$x` erneut aus (verändert).

14 Rekursion

Schreiben Sie eine rekursive Funktion `factorial($n)`, die die Fakultät einer Zahl berechnet ($n! = n * (n-1) * \dots * 1$, mit `factorial(1)` bzw.

`factorial(0)` als Abbruchbedingung, die 1 zurückgibt). Testen Sie die Funktion mit `factorial(5)` (Ergebnis: 120) und mit `factorial(1)`.

15 Anonyme Funktion (Closure)

Legen Sie eine Variable `$discount = 0.1;` an. Schreiben Sie eine anonyme Funktion, die einem Preis den Rabatt abzieht und dabei `$discount` mit `use` einbindet, und speichern Sie sie in einer Variable `$applyDiscount`. Testen Sie sie mit zwei verschiedenen Preisen. Verwenden Sie ausserdem eine anonyme Funktion zusammen mit `array_map()`, um zu einem Array von Zahlen `[1, 2, 3, 4]` ein Array der jeweiligen Quadratzahlen zu berechnen.

16 Arrow Function

Lösen Sie Aufgabe 15 ein zweites Mal - dieses Mal mit **Arrow Functions** (`fn(...) => ...`) statt mit anonymen Funktionen. Vergleichen Sie im Kopf die Schreibweise: Was fällt bei der Arrow Function im Vergleich zur anonymen Funktion weg?

17 Bonusaufgabe: Warenkorb mit Funktionen

Schreiben Sie drei Funktionen: `calculateSubtotal($price, $quantity)` berechnet den Zwischenbetrag eines Produkts, `formatCurrency($amount)` formatiert einen Betrag mit zwei Nachkommastellen und dem Zusatz "CHF" (Tipp: `number_format()`), und `calculateShipping($total)` gibt 0 zurück, wenn `$total` mindestens 15 beträgt, sonst 5.90. Legen Sie ein mehrdimensionales Array `$cart` mit mindestens drei Produkten (`name, price, quantity`) an. Durchlaufen Sie es mit `foreach`, berechnen Sie mit Ihren Funktionen pro Produkt den Zwischenbetrag und geben Sie eine Zeile im Format `Äpfel: 6 x 0.5 = 3.00 CHF` aus. Summieren Sie den Gesamtbetrag, geben Sie Zwischensumme, Versandkosten und den Gesamtbetrag inklusive Versand formatiert mit `formatCurrency()` aus.